בס"ד

מצגת סיכום מינ"פ 1 - מינ"פ 2 מבוא להנדסת תוכנה

אפרת ברינקמן & נעמי גרונשטיין

יולי 2025

הסצנה שלנו

הסצנה מציגה חלל מלא בגופים גאומטרים. כשהמרכזי שבהם היא תחנת הקמח עליה השקענו הרבה מאמצים, והמורכבת מכנפיים משולשות תלת מימדיות, מוקפת בגופים שונים תלת מימדיים חלקם מנסים לדמות את 3 מגדלי עזריאלי המוכרים, כל אלא מוצבים על שטח ורקע כהה עם תאורות המשתקפות על פניהם בצורות השונות.

גאומטריות:

- בסיס התחנה העשוי מגליל סופי צהוב
- גג התחנה עשוי מכדור בעל קוטר, צבע ומקדמי שקיפות זהים לבסיס
- טורבינת התחנה מורכבת מ- 4 פרמידות עם בסיס מרובע שקופות חלקית בצבע סגול
- כדור קטן לסימון מרכז הטורבינה.
- גליל סופי, תיבה מוארכת וב פירמידות בעלות בסיס משולש (דימוי למגדלי עזריאלי)
- פירמידה אדומה שקופה חלקית.
- חרוט כחול סגול
- שלושה כדורים: אחד קריסטלי שקוף, אחד מבריק כהה ומשקף, ואחד בצבעי כנפי התחנה חצי שקוף.

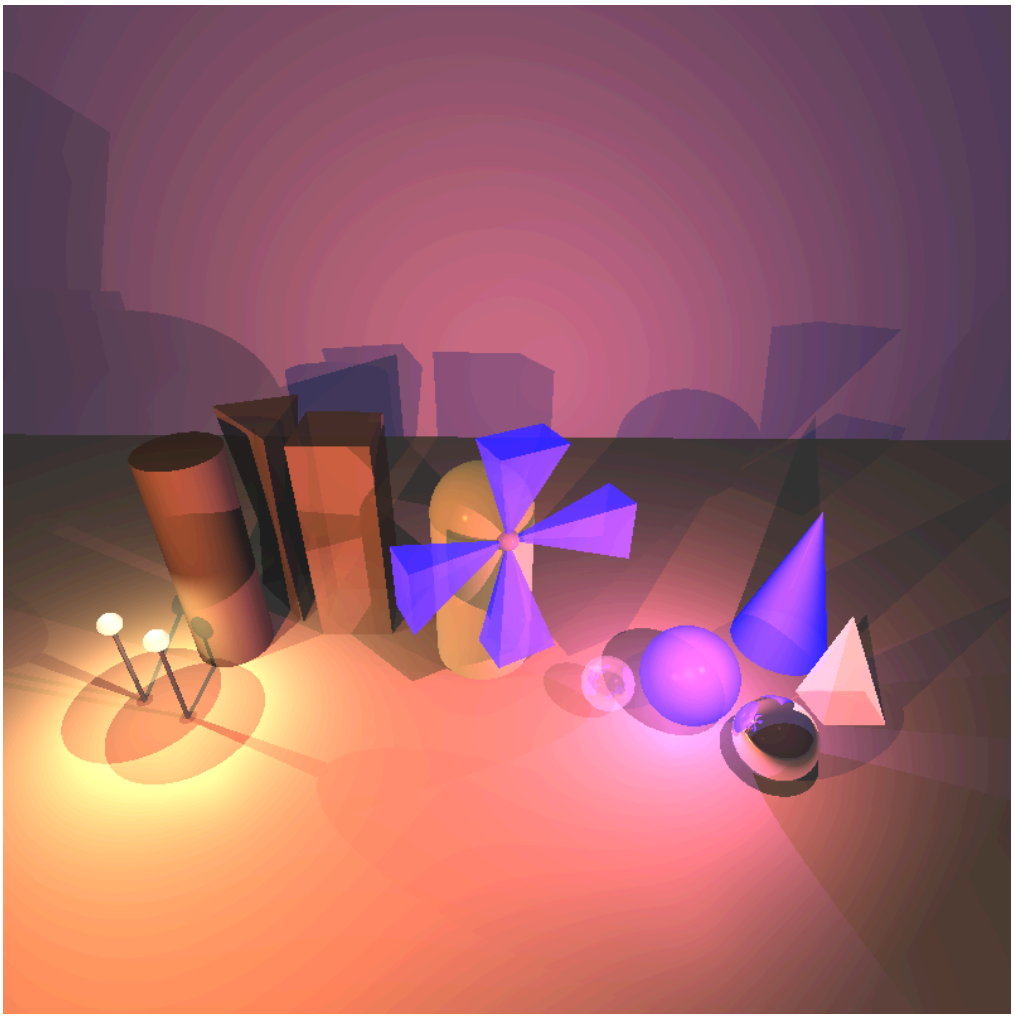
תאורות:

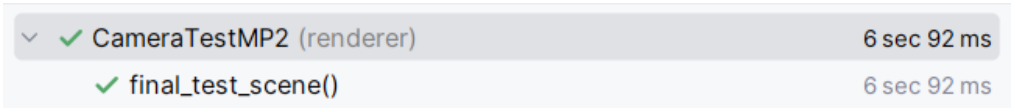
- תאורת ספוט מצד ימין: המדמה שמש בשקיעה
- תאורת ספוט מצד שמאל
- נקודת תאורה מתוך מרכז הכדור הכחול - מזריח אותו
- 2 תאורות ספוט : מדמות 2 פנסי רחוב

מאפיני רינדור כלליים:

- רזולוצייה : 800×800 פיקסלים
- גודל התמונה : 500×500

התמונה לפני השיפורים:





שיפורים

מיני פרויקט 1: Anti-Aliasing

בעיה: רינדור סטנדרטי עם קרן אחת לכל פיקסל יוצר תופעה של קצוות משוננים, בעיקר בקצוות ובאזורים עם
ניגודיות גבוהה. זה יוצר אפקט של מדרגות שפוגע באוטנטיות של התמונה

שיפור: Anti-Aliasing (AA) using Jittered Sampling.

לכל פיקסל נשלחות קרניים רבות למיקומים אקראיים בתוך רשת של פיקסלים (גודל רשת מוגדרת מראש 9*9) , וצבע הפיקסל מחושב כממוצע הצבעים של כל הקרניים. זה מוביל למעברים חלקים יותר ולהפחתת קצוות משוננים.

עיצוב ארכיטקטוני:

למחלקת Camera הוספנו:

שדות חדשים:

```java
/**
 * Flag for enabling anti-aliasing (AA).
 * If true, the camera will use anti-aliasing techniques.
 */
private boolean AA_FLAG = false;  4 usages

/**
 * num of minis pixels at width and height
 */
private int AA_GRID_SIZE = 9;  4 usages
```

פונקצית סט לעידכון הפעלה או כיבוי (במחלקת Builder המוכלת בCamera) :

```java
public Builder setAAFlag(boolean flag) {
    camera.AA_FLAG = flag;
    return this;
}
```

הפונקציה private void castRaysAA(int nX, int nY, int j, int i) :

- שולחת את כל הקרניים שנבנו ע"י constructRaysAA (...) ל- ray tracer.
- מחשבת את ממוצע הצבעים ומציירת את הפיקסל בהתאם.

```java
private void castRaysAA(int nX, int nY, int j, int i) {  3 usages
    //list of rays for the pixel (j, i)
    List<Ray> rays = constructRays(nX, nY, j, i);

    Color color = Color.BLACK;
    for (Ray ray : rays) {
        color = color.add(rayTracer.traceRay(ray));
    }
    color = color.scale( k: 1.0 / rays.size());
    imageWriter.writePixel(j, i, color);
    // If multithreading is enabled, notify the pixel manager that the pixel is done
    if (pixelManager != null)
        pixelManager.pixelDone();
}
```

הפונקציה public List<Ray> constructRaysAA(int nX, int nY, int j, int i) :

- מחשבת רשימה של קרניים לפיקסל ספציפי
- משתמשת בRandom וב AA_GRID_SIZE כדי ליצור פיזור בתוך הפיקסל.
- AA_GRID_SIZE שולט על מספר הקרניים, אם ערכו לדוג 9 הפיזור יהיה(9×9 = 81קרניים)
- שליחת קרניים נשלטת על ידי AA_FLAG בשהוא מופעל משתמשים ב castRaysAA ובשלא ב castRay

```java
public List<Ray> constructRaysAA(int nX, int nY, int j, int i) {  1 usage
    //list of rays for the pixel (j, i)
    List<Ray> rays = new ArrayList<>();
    Random random = new Random();
    //
    double Ry = height / nY;
    double Rx = width / nX;
    //calculate the new width and height of the mini pixel
    double stepY = Ry / AA_GRID_SIZE;
    double stepX = Rx / AA_GRID_SIZE;

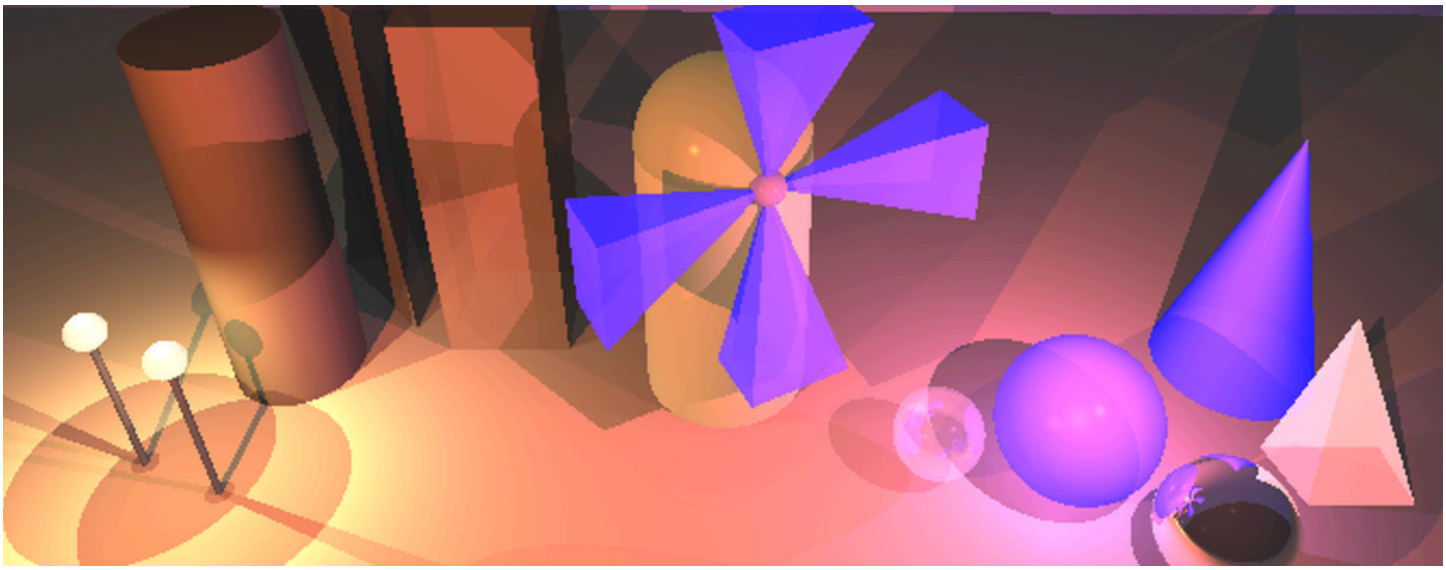
    // Calculate the rays for the pixel (j, i)
    for (int subI = 0; subI < AA_GRID_SIZE; subI++) {
        for (int subJ = 0; subJ < AA_GRID_SIZE; subJ++) {//in mini pixel
            //random offset between 0 and 1
            double jitterY = random.nextDouble(); // Random offset in Y direction
            double jitterX = random.nextDouble(); // Random offset in X direction
            // Calculate the offsets for the sub-pixel location
            double offsetI = (subI + jitterY) * stepY;
            double offsetJ = (subJ + jitterX) * stepX;
            // Calculate the coordinates of the pixel center
            double Yi = -(i - (nY - 1) / 2d) * Ry + offsetI;
            double Xj = (j - (nX - 1) / 2d) * Rx + offsetJ;
            Point pIJ = pCenter;
            if (!isZero(Xj)) pIJ = pIJ.add(vRight.scale(Xj));
            if (!isZero(Yi)) pIJ = pIJ.add(vUp.scale(Yi));
            // Create a ray from the camera position to the sub-pixel location
            Ray ray = new Ray(p0, pIJ.subtract(p0));
            // Add the ray to the list of rays
            rays.add(ray);
        }
    }
    return rays;
}
```

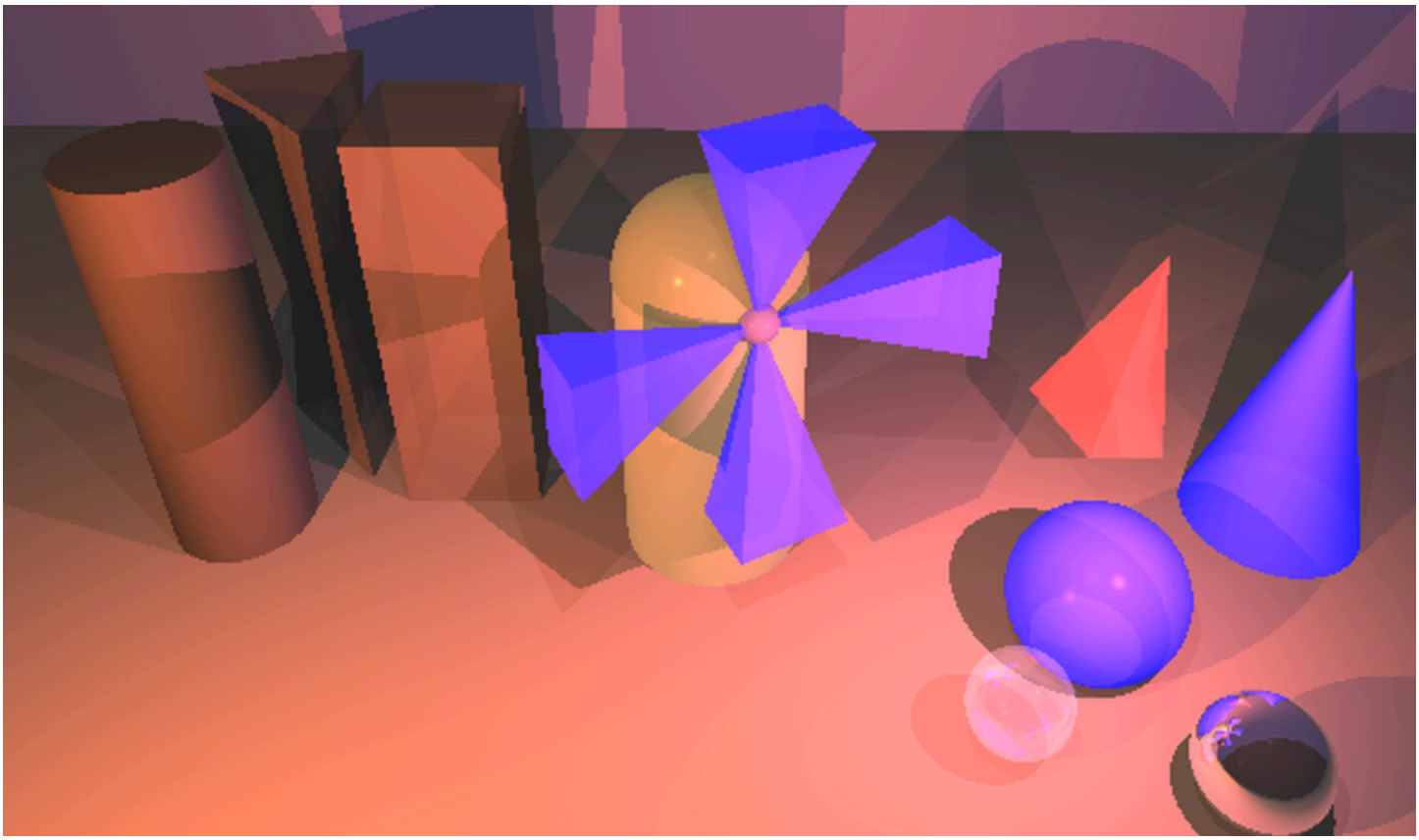
תוצאות:

לפני שיפור AA :

setAAFlag(false)

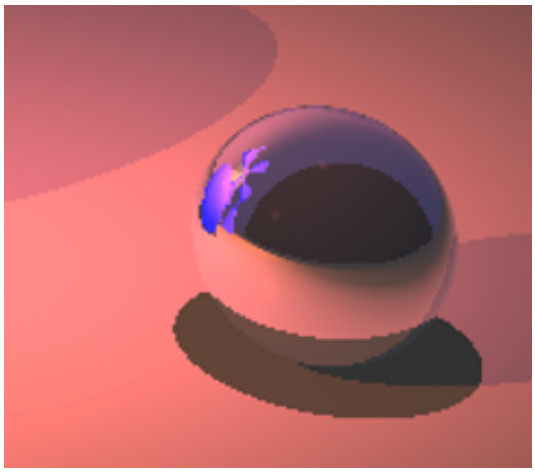
בקצוות של הגופים בתמונות, נראים לעיתים קרובות "שיניים משוננות" (aliasing).





CameraTestMP2 (renderer) 6 sec 92 ms

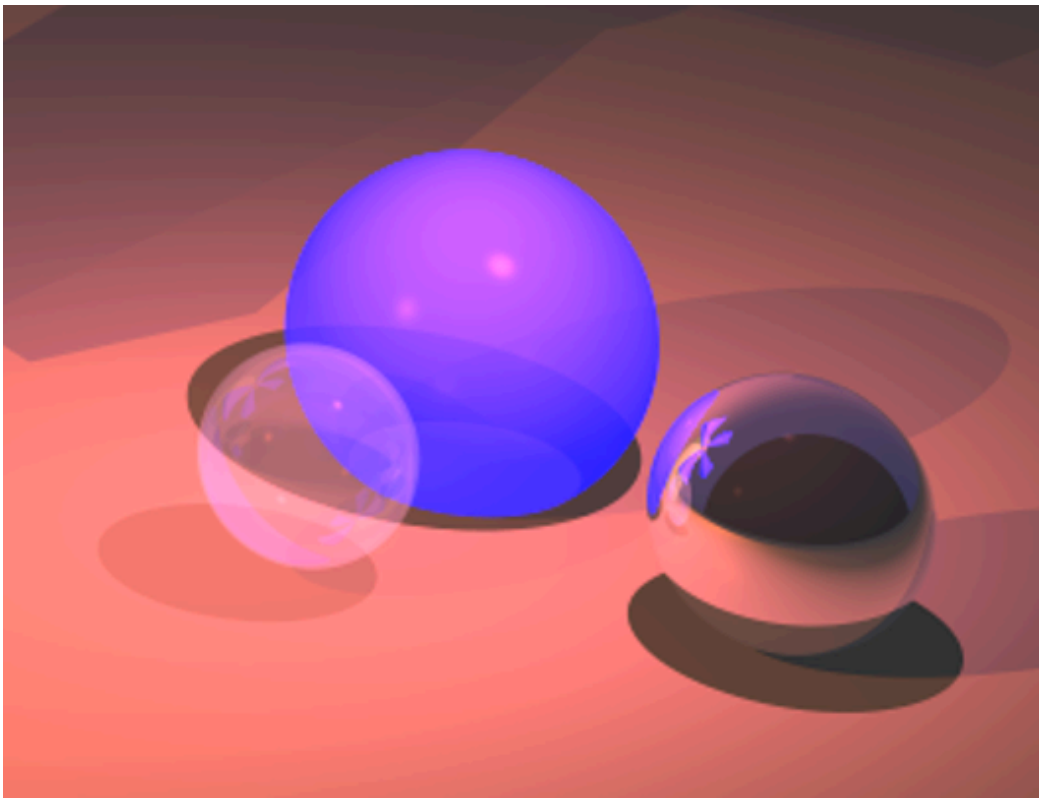
final_test_scene() 6 sec 92 ms

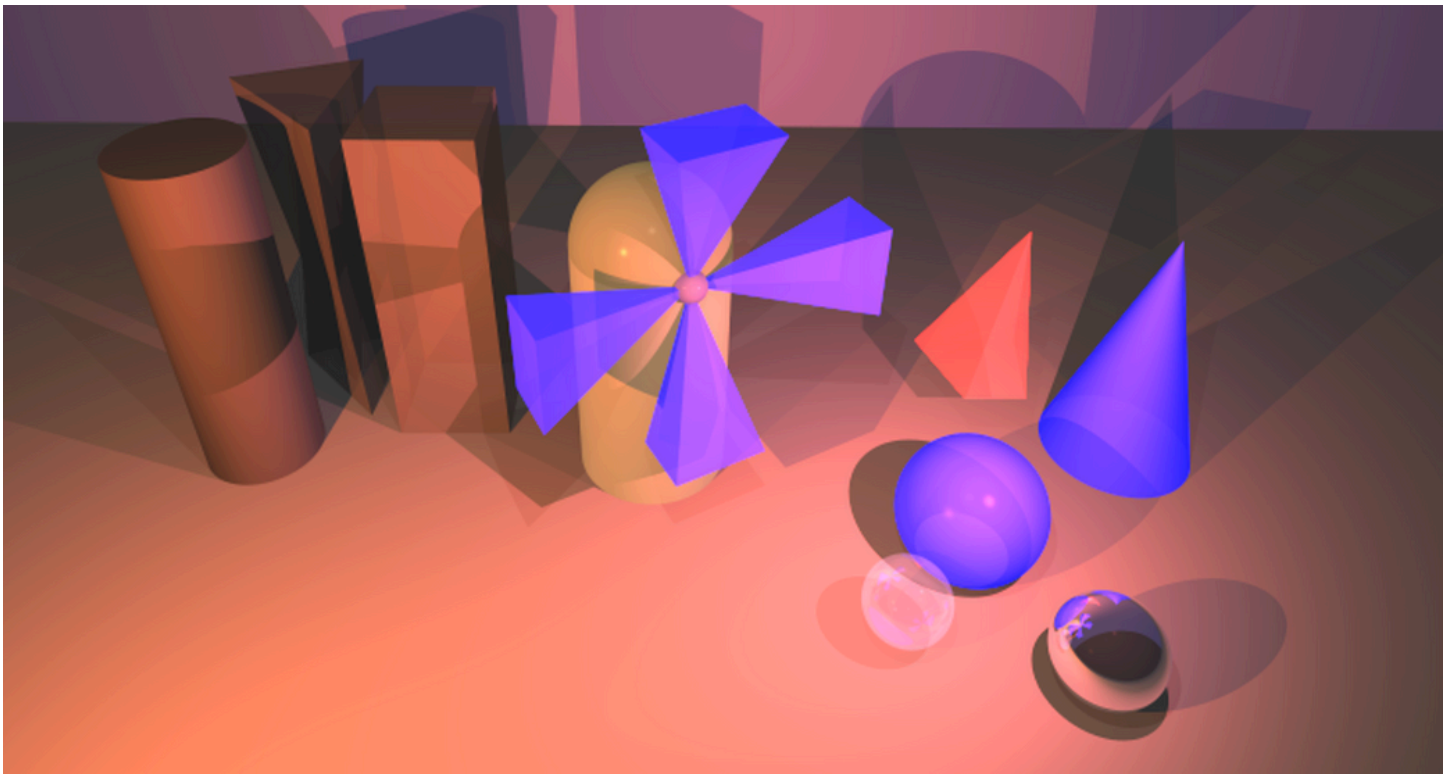


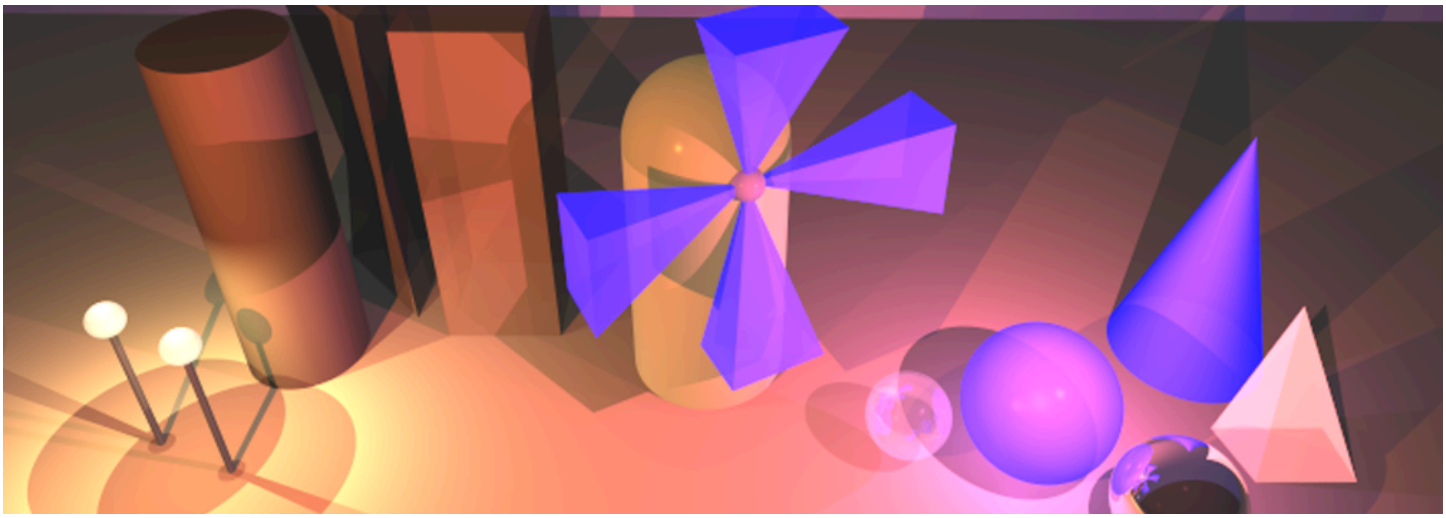
לאחר שיפור AA :

setAAFlag(true)

קווים חלקים, הרבה פחות מפוקסל.









השפעה:

קצוות הגופים היו עם מדרגות בקצוות שפוגעות בנראות, לאחר השיפור המדרגות הוחלקו והתמונה נראת חדה יותר

למה זה עובד טוב

רינדור פיזיקלי של זכוכית = אפקטים אופטיים אמיתיים

דגימה אדפטיבית חוסכת משאבים באזורים אחידים

המערכת המודולרית מאפשרת שליטה קלה על תכונות החומרים

זמן הריצה גדל מ6~ שניות ל 8~ דקות!

מיני פרויקט 2: Adaptive Super Sampling

בעיה: בשיפור של AA שלחנו קרניים רבות באופן אחיד לכל פיקסל – גם באזורים שבהם אין בכך צורך. כתוצאה מכך, מתבזבזים משאבים חישוביים על אזורים חלקים שאינם דורשים רמת פירוט גבוהה

שיפור: Adaptive Super Sampling

- שולחים קרן למכז הפיקסל ומחלקים אותו ל4 חלקים
- שולחים קרניים לכל רבע, עבור כל תת־פיקסל :

  מחשבים שונות בצבעים

  אם השונות נמוכה, עוצרים ומחשבים ממוצע
  אם השונות גבוהה, ממשיכים לחלק את הפיקסל

עיצוב ארכיטקטוני

למחלקת Camera הוספנו:

שדות חדשים:

```
/**
 * Flag for enabling adaptive super-sampling
 * If true, the camera will use adaptive super-sampling techniques.
 */
private boolean ASS_FLAG = false;  4 usages

/**
 * LEVEL is the recursion level for adaptive super-sampling
 */
private int LEVEL = 2;  1 usage
```

פונקצית סט לעדכון הפעלה או כיבוי (במחלקת Builder המוכלת בCamera):

```
public Builder setASSFlag(boolean flag) {
    camera.ASS_FLAG = flag;
    return this;
}
```

הפונקציה private void castRayASS(int nX, int nY, int j, int i):

יוצרת קרן למרכז הפיקסל ומפעילה את הפונקציה הרקורסיבית

```
private void castRayASS(int nX, int nY, int j, int i) {  3 usages   naomi1877 *
    double pixelWidth = width / nX;
    double pixelHeight = height / nY;

    // Calculate the center of the pixel
    double xJ = (j - (nX - 1) / 2.0) * pixelWidth;
    double yI = -1 * (i - (nY - 1) / 2.0) * pixelHeight;

    Point pIJ = pCenter;
    if (!isZero(xJ)) pIJ = pIJ.add(vRight.scale(xJ));
    if (!isZero(yI)) pIJ = pIJ.add(vUp.scale(yI));

    // level is the recursion level for adaptive super-sampling
    Color color = adaptiveSuperSampling(pIJ, pixelWidth, pixelHeight, LEVEL);
    imageWriter.writePixel(j, i, color);

    if (pixelManager != null)
        pixelManager.pixelDone();
}
```

הפונקציה הרקורסיבית:

private Color adaptiveSuperSampling(Point center, double pixelWidth, double pixelHeight, int level)

שדוגמת את מרכז הפיקסל ובודקת כמה הצבעים בשאר הפיקסל שונים ממנה ורק באשר השונות גבוהה תופעל הפונקציה באופן רקורסיבית ותפצל את רבע הפיקסל שוב

תיאור גיאומטרי

השיטה פועלת על ריבועי פיקסלים, ומחלקת שוב רק באזורים שבהם יש שונות גבוהה בצבע. כך מושגת יעילות גבוהה: אזורים פשוטים כמעט ואינם מחולקים, ואילו אזורים מורכבים מקבלים יותר קרניים לדיוק מרבי.

```
private Color adaptiveSuperSampling(Point center, double pixelWidth, double pixelHeight, int level) {  2 usages  naom
    if (level == 0) {
        return rayTracer.traceRay(new Ray(p0, center.subtract(p0)));
    }

    double halfWidth = pixelWidth / 2.0;
    double halfHeight = pixelHeight / 2.0;
    Random random = new Random();

    // Calculate the color of center point of the pixel
    Color totalColor = rayTracer.traceRay(new Ray(p0, center.subtract(p0)));
    int count = 1; //

    //the four corners of the pixel
    double[] xSigns = {-1, 1, -1, 1};
    double[] ySigns = {1, 1, -1, -1};
    // Create sample points in the four corners of the pixel
    for (int k = 0; k < 4; k++) {
        double xSign = xSigns[k];
        double ySign = ySigns[k];

        double randX = random.nextDouble() * halfWidth / 2;
        double randY = random.nextDouble() * halfHeight / 2;

        Point samplePoint = center
                .add(vRight.scale( rhs: xSign * (halfWidth / 2 + randX)))
                .add(vUp.scale( rhs: ySign * (halfHeight / 2 + randY)));

        Color sampleColor = rayTracer.traceRay(new Ray(p0, samplePoint.subtract(p0)));


        if (isColorCloseEnough(totalColor, sampleColor)) {
            totalColor = totalColor.add(sampleColor);
            count++;
        } else {
            // recursively sample the quarter
            Point quarterCenter = center//Calculate the center of the quarter
                    .add(vRight.scale( rhs: xSign * halfWidth / 2))
                    .add(vUp.scale( rhs: ySign * halfHeight / 2));

            Color recursiveColor = adaptiveSuperSampling(quarterCenter, halfWidth, halfHeight,  level: level - 1);
            totalColor = totalColor.add(recursiveColor);
            count++;
        }
    }

    return totalColor.scale( k: 1.0 / count);
}
```

הפונקציה private boolean isColorCloseEnough(Color c1, Color c2):

נקראת על ידי הפונקציה adaptiveSuperSampling, בודקת את רמת הדמיון בין 2 צבעים ומחזירה האם מספיק דומים או לא לפי רמת הרגישות המוגדרת מראש במשתנה: THRESHOLD

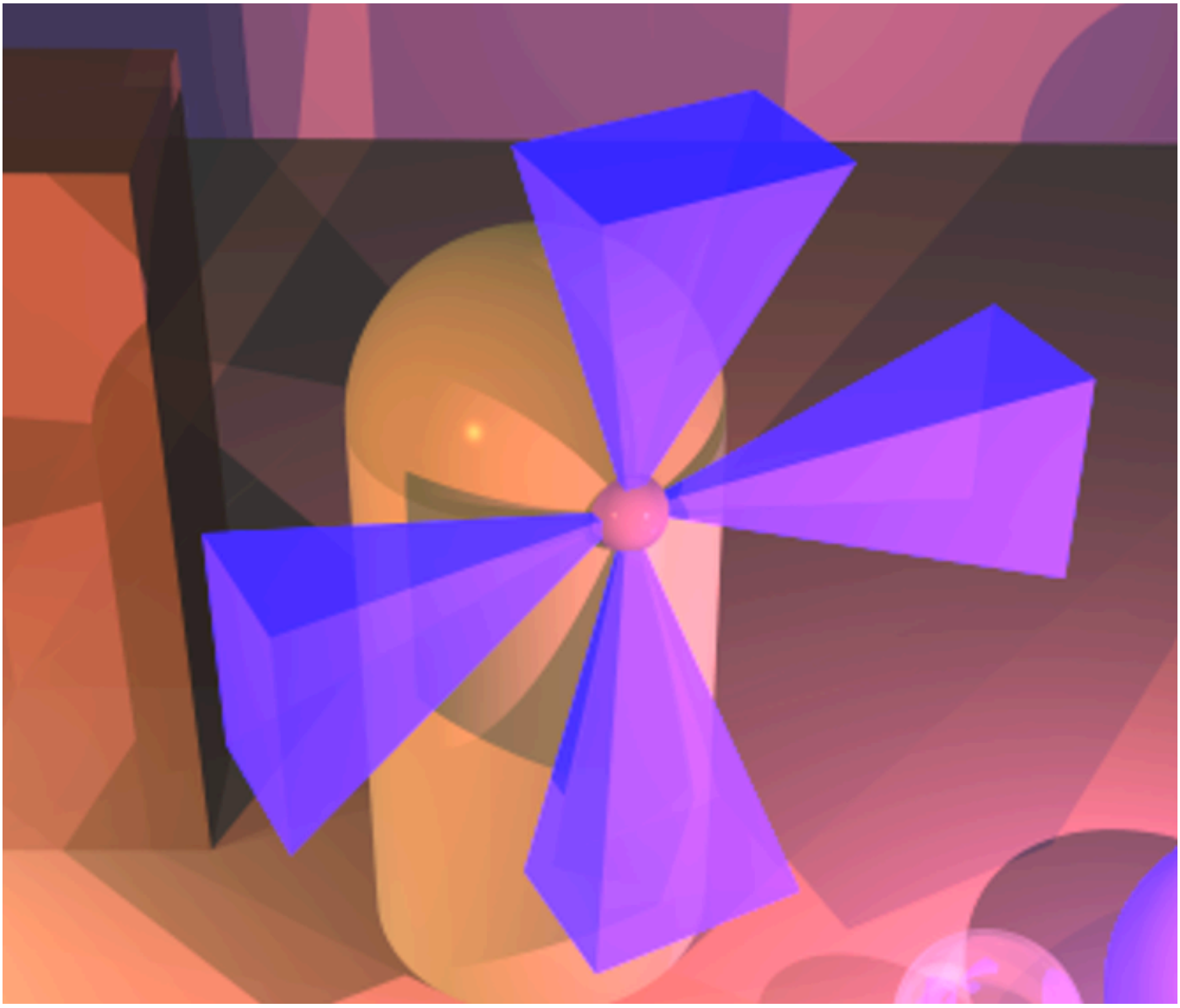
```
private boolean isColorCloseEnough(Color c1, Color c2) {
    final double THRESHOLD = 0.1;

    Double3 diff = c1.getRgb().subtract(c2.getRgb());
    return Math.abs(diff.d1()) < THRESHOLD &&
            Math.abs(diff.d2()) < THRESHOLD &&
            Math.abs(diff.d3()) < THRESHOLD;
}
```

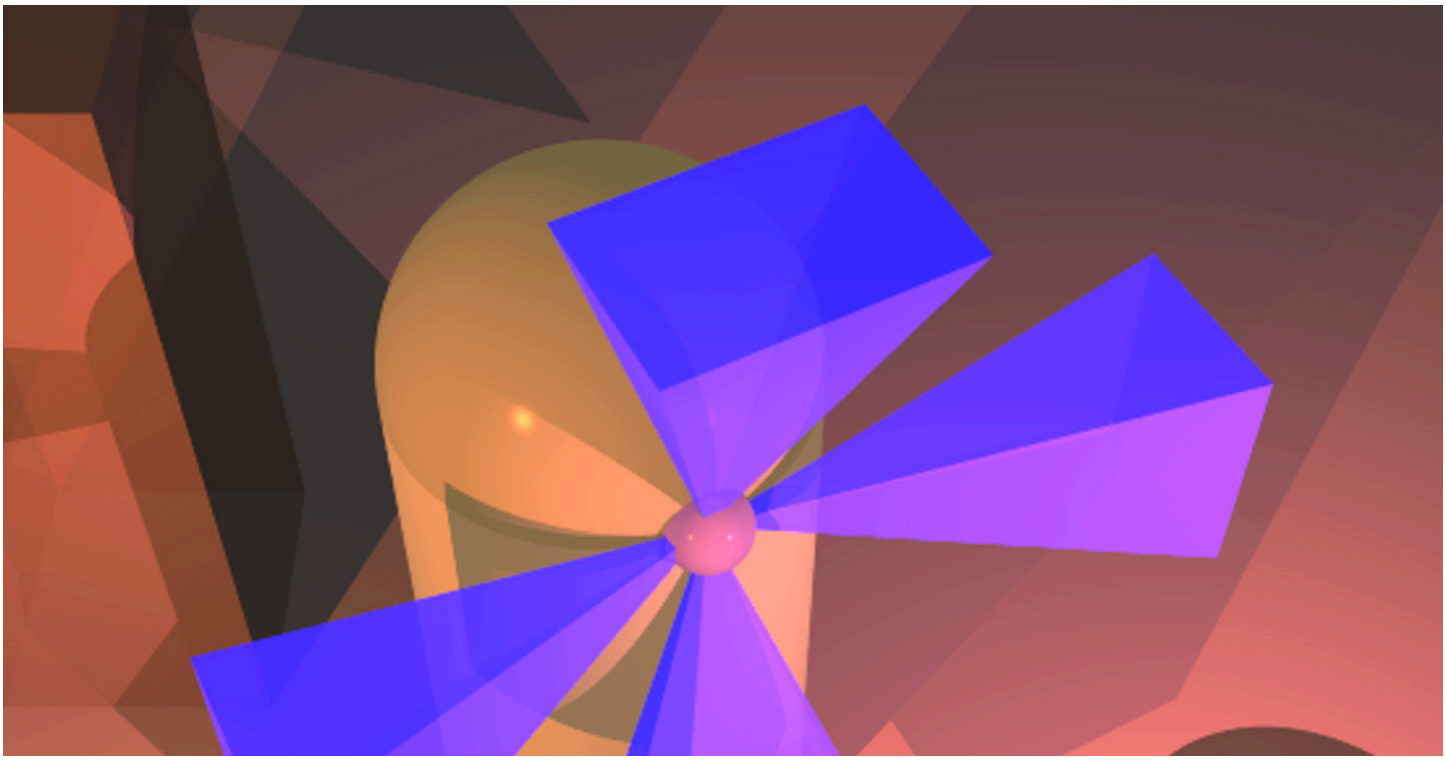
תוצאות:

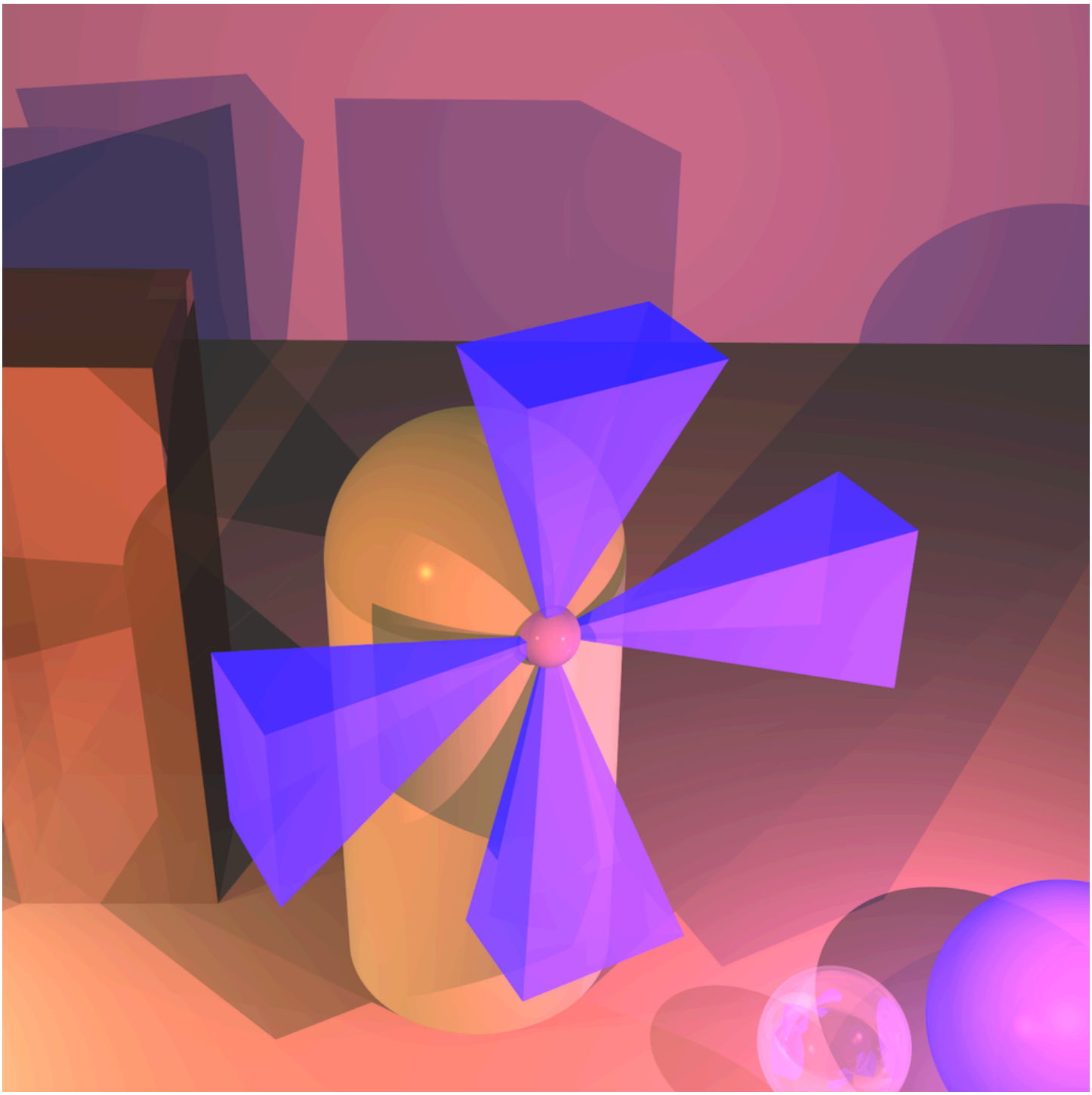
לפני שיפור ASS:

← כפי שראינו בתמונות עם שיפור AA



לאחר שיפור ASS:
setASSFlag(true)
setMultithreading(8)





✓ CameraTestMP2 (renderer) 4 min 38 sec

✓ final_test_scene() 4 min 38 sec

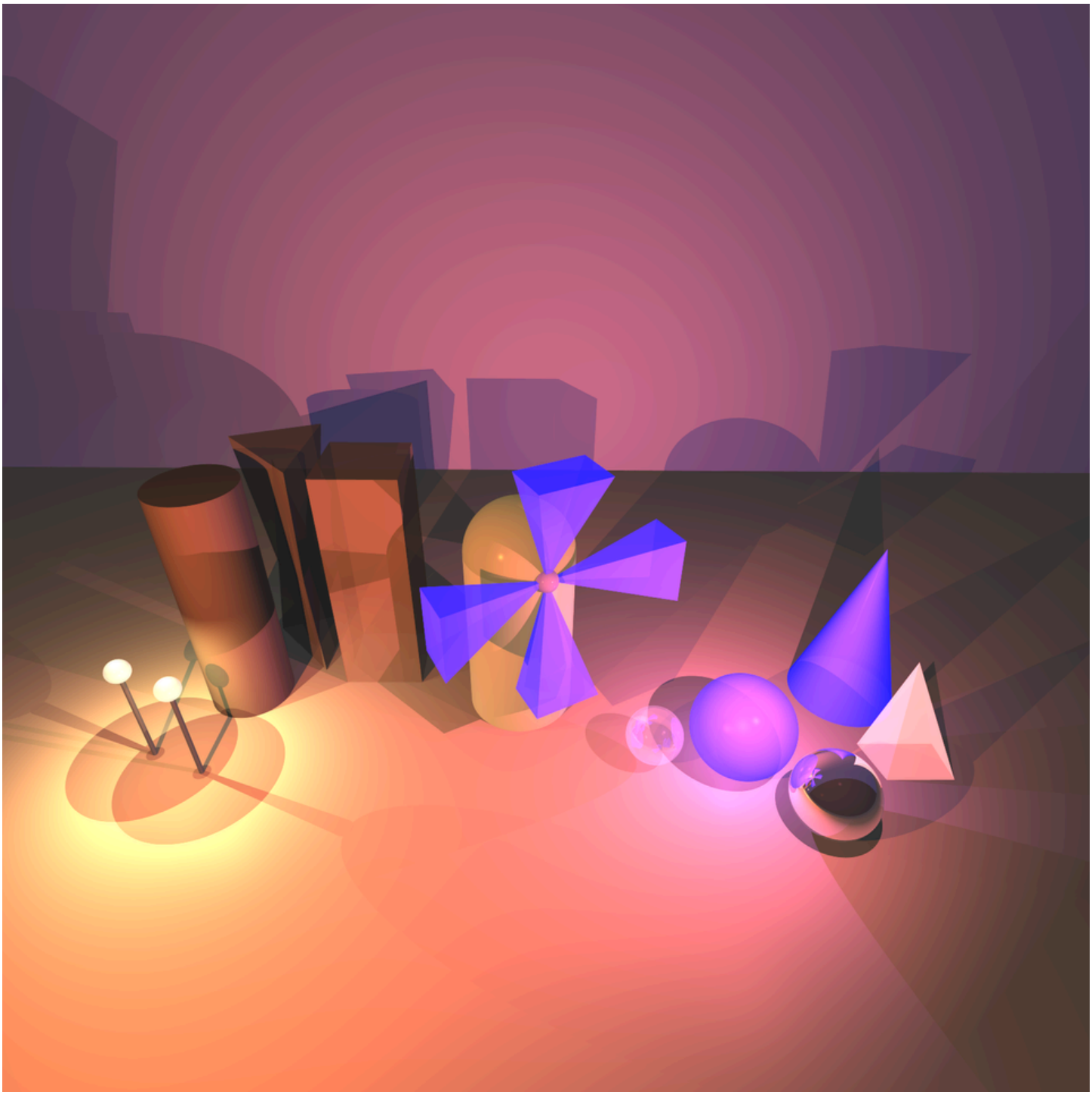
- פינות חדות יותר
- אזורים אחידים מקבלים פחות קרניים - שיפור ביצועים

תמונה סופית:

כולל ריבוי תהליכונים

setASSFlag(true)

setMultithreading(-2)



✓ CameraTestMP2 (renderer)	1 min 42 sec
✓ final_test_scene()	1 min 42 sec

השפעה:

למה זה עובד טוב

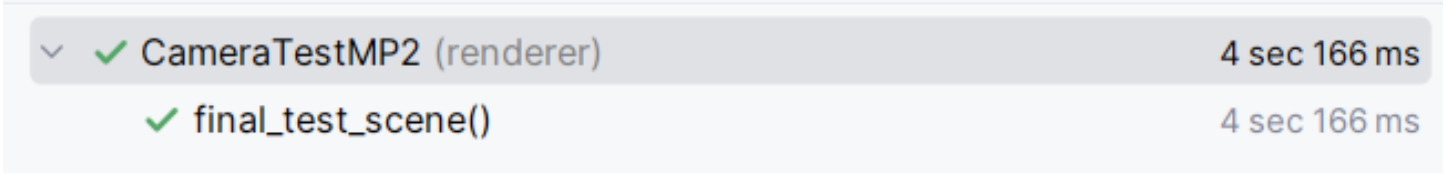
יעיל: מתמקד באזורים שדורשים יותר דיוק

דינמי: מתאים את עצמו למורכבות

איכות גבוהה: קצוות חלקים ושמירה על פרטים

סיכום זמני ביצועים:

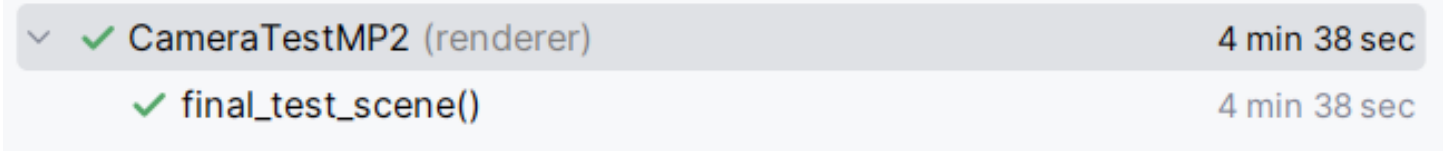
ללא שיפור AA:



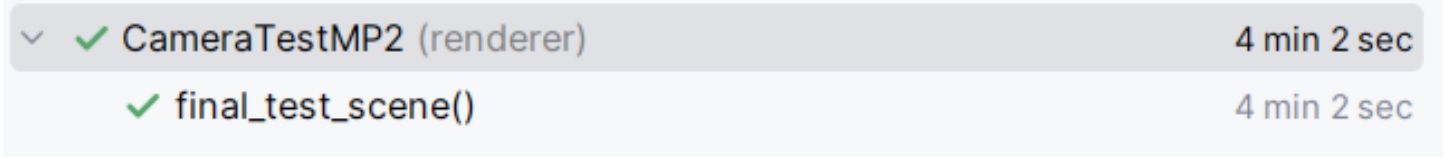
יחד עם AA:



יחד עם שיפור ASS (ראשוני= ללא ריבוי תהליכונים)



AA+multithreding(-2):



ASS+multithreding(-2):



בונוסים:

- נורמל לגליל סופי
- חיתוך עם גליל אינסופי
- חיתוך עם מצולע
- חיתוך עם גליל סופי
- שימוש בקבצי XML
- תמונה עם 10+ עצמים לשלב 7
- Jitter עבור AA וכן ASS
- הוספת מחלקות גופים:

1. Cone // נעזרנו בGPT על מנת לממש את פונקצית מציאת החיתוכים ופונקצית הנורמל.
2. Pyramid
3. Box

במימוש הגופים Box ו־Pyramid עשינו שימוש בתבנית Composite:
כל גוף מורכב מחלקים פשוטים (פאות), והם מאוגדים באובייקט Geometries, כך שהמנוע הגרפי מתייחס אליהם כגוף אחד, תוך שימוש בקוד אחיד לחישובי חיתוך ונורמל.

תיבה (Box)

- מיוצגת על־ידי שש פאות מלבניות (Polygon).
- כל פאה נוצרת לפי מיקום הנקודות הקצה – min, max.
- נשמרת רשימת הפאות sideFaces.
- כל הפאות מאוחדות במבנה Geometries לצורך חישובי חיתוך ונורמל.

פירמידה (Pyramid)

- מיוצגת על־ידי:
  - בסיס אחד (Polygon)
    - משולשים צדדיים (Triangle) לכל זוג נקודות רצופות בבסיס.
- כל המשולשים מחוברים לנקודת הקודקוד top.
- גם כאן – כל הפאות נשמרות ב־Geometries.

יתרונות המימוש:

- **פשטות** – לכל פאה חישובי חיתוך ונורמל ידועים מראש.
- **שימוש חוזר בקוד** – באמצעות מחלקות קיימות (Triangle, Polygon).
- **אחידות בהתנהגות** – תומך באותן פעולות: חומר, צבע, חישוב נורמל וחיתוך.

מימוש פונקציית public Vector getNormal(Point point) של הגופים Pyramid ו־Box:

נדרש לזהות על איזו פאה נמצאת הנקודה.
לשם כך, הוספנו למחלקת Polygon את הפונקציה public boolean isOn(Point point):

- בודקת שהנקודה נמצאת על מישור הפאה.
- בודקת גאומטרית שהנקודה נמצאת בתוך תחום הפאה.

```java
public boolean isOn(Point point) {  3 usages  naomi1877
    // Step 1: Check if point is on the same plane
    Vector n = this.getNormal(point);
    // using one of the polygon's base points
    Vector qToP = point.subtract(this.vertices.getFirst());
    double distance = alignZero(n.dotProduct(qToP));
    if (!isZero(distance)) {
        // point is not on the plane
        return false;
    }
    // Step 2: Check if the point is inside the polygon
    int size = this.vertices.size();
    Vector v1, v2;
    for (int i = 0; i < size; ++i) {
        try {
            v1 = this.vertices.get(i).subtract(point);
            v2 = this.vertices.get((i + 1) % size).subtract(point);
        } catch (IllegalArgumentException e) {
            // in case the point is on a vertex - skip
            continue;
        }
        try {
            Vector cross = v1.crossProduct(v2);
            if (alignZero(cross.dotProduct(n)) < 0) {
                return false; // point is outside
            }
        } catch (IllegalArgumentException e) {
            // points are collinear, skip this edge
            return false;
        }
    }
    // point is on the plane and inside the polygon
    return true;
}
```

הפונקציה בשני המחלקות:

```java
@Override
protected List<Intersection> calculateIntersectionsHelper(Ray ray) {
   return components.calculateIntersections(ray);
}
```

```java
public class Pyramid extends Geometry {
    private final Point top;
    private final Polygon base;
    private final List<Triangle> sideFaces = new LinkedList<>();
    private final Geometries components = new Geometries();

    public Pyramid(Point top, Polygon base) {
       this.top = top;
       this.base = base;

       List<Point> vertices = base.vertices;
       if (vertices.size() < 3)
```

```
            throw new IllegalArgumentException("Base must have at least 3 vertices");

        // saving the contour of the piramid base
        for (int i = 0; i < vertices.size(); i++) {
            Point p1 = vertices.get(i);
            Point p2 = vertices.get((i + 1) % vertices.size());
            sideFaces.add(new Triangle(top, p1, p2));
        }

        // adding the triangles and base to the components
        components.add(base);
        components.add(sideFaces.toArray(new Triangle[0]));
    }
.
.
.
}


public class Box extends Geometry{
    private final Double3 min;
    private final Double3 max;
   private final List<Polygon> sideFaces;
    private final Geometries components = new Geometries();

    public Box(Double3 p1, Double3 p2 ) {
        if (p1.d1()==p2.d1() ||
            p1.d2()==p2.d2() ||
            p1.d3()==p2.d3()) {
            throw new IllegalArgumentException("Points must not be aligned on the same axis");
        }
        this.min = new Double3( Math.min(p1.d1(),p2.d1()),Math.min(p1.d2(),p2.d2()),Math.min(p1.d3(),p2.d3()));
        this.max = new Double3( Math.max(p1.d1(),p2.d1()),Math.max(p1.d2(),p2.d2()),Math.max(p1.d3(),p2.d3()));
        this.sideFaces=List.of( // Left (x = minX) – counter-clockwise when looking from x < 0
                new Polygon(
                        new Point(min.d1(), min.d2(), max.d3()),
                        new Point(min.d1(), max.d2(), max.d3()),
                        new Point(min.d1(), max.d2(), min.d3()),
                        new Point(min.d1(), min.d2(), min.d3())
                ),
                // Right (x = maxX) – counter-clockwise when looking from x > 0
                new Polygon(
                        new Point(max.d1(), min.d2(), min.d3()),
                        new Point(max.d1(), max.d2(), min.d3()),
                        new Point(max.d1(), max.d2(), max.d3()),
                        new Point(max.d1(), min.d2(), max.d3())
                ),
```

```
            // Bottom (y = minY) - counter-clockwise from y < 0
            new Polygon(
                    new Point(min.d1(), min.d2(), max.d3()),
                    new Point(max.d1(), min.d2(), max.d3()),
                    new Point(max.d1(), min.d2(), min.d3()),
                    new Point(min.d1(), min.d2(), min.d3())
            ),
            // Top (y = maxY) - counter-clockwise from y > 0
            new Polygon(
                    new Point(min.d1(), max.d2(), max.d3()),
                    new Point(max.d1(), max.d2(), max.d3()),
                    new Point(max.d1(), max.d2(), min.d3()),
                    new Point(min.d1(), max.d2(), min.d3())
            ),
            // Front (z = minZ) - counter-clockwise from z < 0
            new Polygon(
                    new Point(min.d1(), min.d2(), min.d3()),
                    new Point(min.d1(), max.d2(), min.d3()),
                    new Point(max.d1(), max.d2(), min.d3()),
                    new Point(max.d1(), min.d2(), min.d3())
            ),
            // Back (z = maxZ) - counter-clockwise from z > 0
            new Polygon(
                    new Point(max.d1(), min.d2(), max.d3()),
                    new Point(max.d1(), max.d2(), max.d3()),
                    new Point(min.d1(), max.d2(), max.d3()),
                    new Point(min.d1(), min.d2(), max.d3())
            ));
    // Create the 6 faces of the box
    components.add( sideFaces.toArray(new Polygon[0]));
  }
.
.
.
}
```

ובונוס אחרון שלא הספקנו לספר עליו : ב"ה נעמי בהריון מתקדם...